

SOLID

Principles

Interview

Guide

By Ramesh Fadatara (Java Guides)

Single Responsibility Principle

- The Single Responsibility Principle (SRP) is one of the five SOLID principles of object-oriented design.
- SRP states that a class should have only one reason to change.
- In simple words, a class should do only one job and handle one responsibility.
- When a class starts doing multiple things, it becomes harder to maintain, harder to test, harder to modify, and very fragile.
- A small change in one part can break something else that is completely unrelated.
- SRP keeps your code clean, modular, and easy to extend without modifying existing functionality.

Why Is the Single Responsibility Principle Important?

- SRP reduces complexity by making classes smaller, cleaner, and easier to understand.
- It improves maintainability because changes affect only one class instead of multiple classes.
- It improves testability since each class handles one thing, making unit tests easier to write.
- SRP improves reusability by separating responsibilities so classes can be reused without unnecessary code.
- Frameworks like Spring, domain-driven design layers, and clean architecture strongly encourage SRP.

Clean Interview Answer:

The Single Responsibility Principle states that a class should have only one reason to change and should handle only one responsibility.

When a class mixes multiple responsibilities, it becomes tightly coupled and difficult to maintain.

By separating responsibilities into focused classes, the system becomes maintainable, flexible, and easy to test.

Open–Closed Principle

- The Open–Closed Principle (OCP) is the second SOLID principle.
- It says that a software module such as a class, method, or component should be open for extension but closed for modification.
- In simple words, you should be able to add new functionality without modifying existing code.
- This protects your code from unexpected side effects.
- When you modify a tested class, something may break.
- But if you extend a class without touching its original logic, you avoid bugs and keep the system stable.
- OCP encourages abstraction, polymorphism, and interface-driven design.

Why Is OCP Important?

- OCP helps you build systems that grow over time without rewriting existing code.
- It increases stability because tested code remains untouched.
- It supports loose coupling because the main class depends on abstractions instead of concrete implementations.
- Frameworks like Spring use OCP everywhere—you extend behavior by plugging in new classes instead of modifying framework code.

Clean Interview Answer:

- The Open–Closed Principle states that software modules should be open for extension but closed for modification.
- You should be able to add new behavior without changing existing tested code.
- This is achieved through abstraction and polymorphism.
- Instead of modifying a class when requirements change, you extend it by creating new implementations.
- This reduces bugs, improves maintainability, and keeps the system scalable.

Liskov Substitution Principle

- The Liskov Substitution Principle (LSP) is the third SOLID principle, introduced by Barbara Liskov, and focuses on correct inheritance design.
- It states that objects of a parent class should be replaceable with objects of its child class without breaking the program.
- In simple words: if B extends A, then B should work anywhere A is used.
- If replacing A with B breaks the logic or changes expected behavior, inheritance was designed incorrectly and LSP is violated.
- The purpose of LSP is to ensure meaningful inheritance where child classes extend behavior rather than break it.
- Example understanding: “If a method expects a Car, and I pass a SportsCar, it should still work normally.”

Why Is LSP Important?

- LSP protects your design from incorrect inheritance.
- It ensures subclasses can be used wherever the parent class is expected, without surprises.
- It prevents hidden bugs that show up only at runtime.
- It encourages interface-based design and proper abstraction.
- It makes polymorphism reliable because substituting child classes will not break existing code.

Clean Interview Answer:

- LSP says that a subclass should be replaceable wherever the parent class is expected without breaking behavior.
- If substituting the child changes outputs, throws exceptions, or violates expectations, inheritance was designed incorrectly.
- The goal of LSP is to ensure safe inheritance and predictable polymorphism.

Interface Segregation Principle

- The Interface Segregation Principle (ISP) is the fourth SOLID principle.
- It states that no client should be forced to depend on methods it does not use.
- Instead of creating one large interface with many unrelated methods, you should create small, focused interfaces that contain only the methods clients actually need.
- An interface should not act like a menu card full of items unrelated to a particular class.
- If a class implements an interface, it should implement only the methods that are relevant to it.
- Large interfaces cause tight coupling, unnecessary method implementations, and code that breaks easily when the interface changes.
- ISP encourages designing interfaces that are clean, small, and highly cohesive.

Why Is ISP Important?

- ISP ensures that interfaces remain clean, focused, and free from unnecessary responsibilities.
- It eliminates fat interfaces that force classes to implement irrelevant methods.
- It prevents fragile implementations that break when new methods are added to an interface.
- It improves testability, reduces coupling, and makes your design more modular.
- Large enterprise frameworks, including Spring, follow ISP by providing small interfaces with minimal methods.

Clean Interview Answer:

- The Interface Segregation Principle states that clients should not be forced to depend on methods they do not use.
- Instead of one large interface, we create multiple small and cohesive interfaces so classes implement only what they need.
- This avoids unused methods, reduces coupling, and makes the system more maintainable.

Dependency Inversion Principle

- The Dependency Inversion Principle (DIP) is the fifth and final SOLID principle.
- DIP says two important things:
 - High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces or abstract classes).
 - Abstractions should not depend on details; details should depend on abstractions.
- In simple words, your classes should depend on interfaces, not concrete implementations.
- Depending on concrete classes makes code tightly coupled, hard to change, and difficult to test.
- Depending on interfaces or abstract classes makes code flexible, modular, and easy to extend without breaking existing logic.

Why Is DIP Important?

- DIP reduces coupling, increases flexibility, and makes systems easier to extend.
- It allows switching implementations without breaking existing code.
- It supports dependency injection, which is the core concept behind frameworks like Spring.
- Unit testing becomes effortless because mocks and stubs can be injected instead of real implementations.
- Entire architectures like Clean Architecture and Hexagonal Architecture rely heavily on DIP.

Clean Interview Answer:

- The Dependency Inversion Principle states that high-level modules should not depend on low-level modules; both should depend on abstractions.
- Instead of depending on concrete classes, we depend on interfaces or abstract classes.
- This makes software flexible, testable, and extendable without modifying existing code.